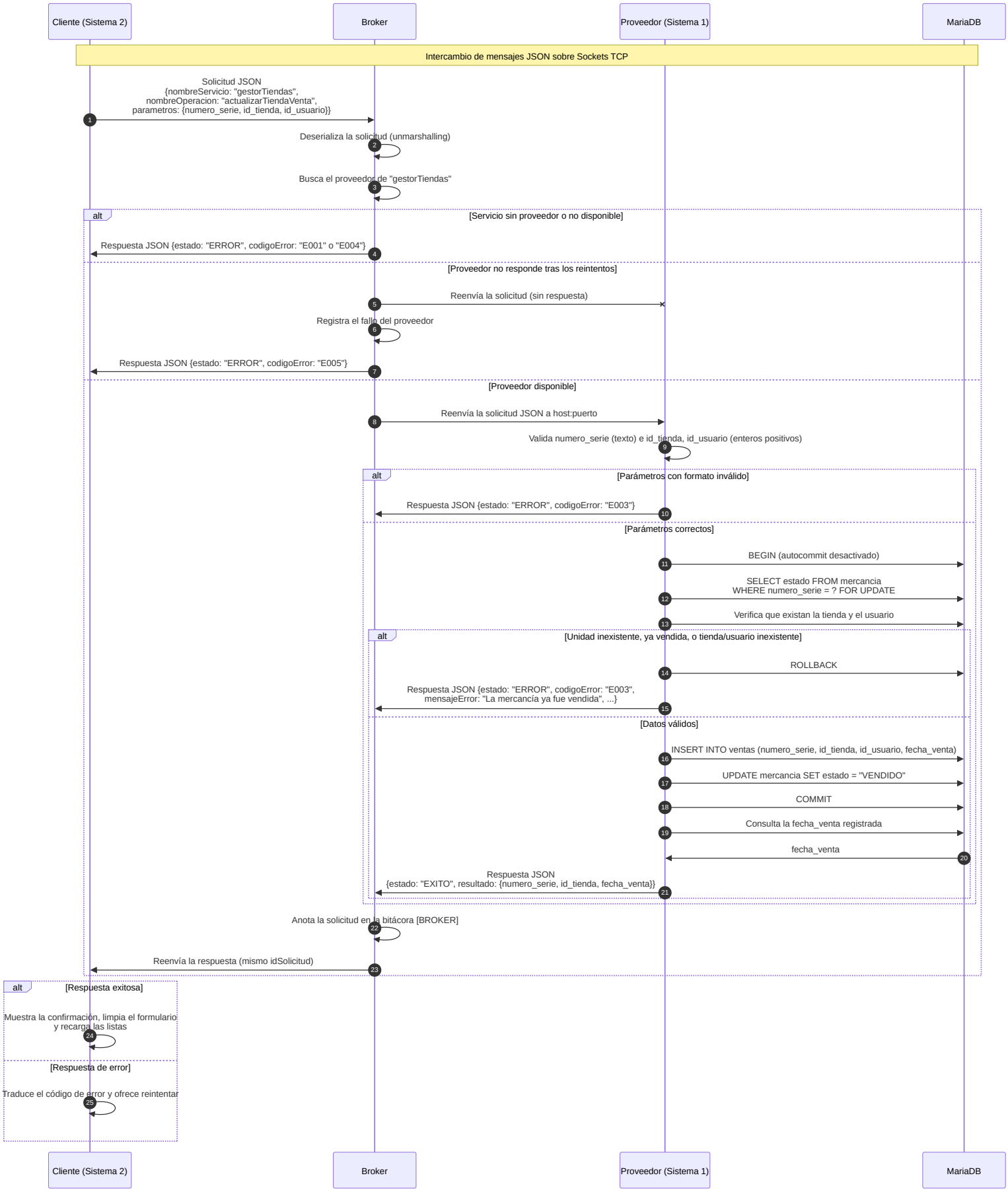

Cliente (Sistema 2)
Broker
Proveedor (Sistema 1)
MariaDB
Intercambio de mensajes JSON sobre Sockets TCP
Solicitud JSON
{nombreServicio: "gestorTiendas",
nombreOperacion: "actualizarTiendaVenta",
parametros: {numero_serie, id_tienda, id_usuario}}
1
Deserializa la solicitud (unmarshalling)
2
Busca el proveedor de "gestorTiendas"
3
alt
[Servicio sin proveedor o no disponible]
Respuesta JSON {estado: "ERROR", codigoError: "E001" o "E004"}
4
[Proveedor no responde tras los reintentos]
Reenvía la solicitud (sin respuesta)
5
Registra el fallo del proveedor
6
Respuesta JSON {estado: "ERROR", codigoError: "E005"}
7
[Proveedor disponible]
Reenvía la solicitud JSON a host:puerto
8
Valida numero_serie (texto) e id_tienda, id_usuario (enteros positivos)
9
alt
[Parámetros con formato inválido]
Respuesta JSON {estado: "ERROR", codigoError: "E003"}
10
[Parámetros correctos]
BEGIN (autocommit desactivado)
11
SELECT estado FROM mercancia
WHERE numero_serie = ? FOR UPDATE
12
Verifica que existan la tienda y el usuario
13
alt
[Unidad inexistente, ya vendida, o tienda/usuario inexistente]
ROLLBACK
14
Respuesta JSON {estado: "ERROR", codigoError: "E003",
mensajeError: "La mercancía ya fue vendida", ...}
15
[Datos válidos]
INSERT INTO ventas (numero_serie, id_tienda, id_usuario, fecha_venta)
16
UPDATE mercancia SET estado = "VENDIDO"
17
COMMIT
18
Consulta la fecha_venta registrada
19
fecha_venta
20
Respuesta JSON
{estado: "EXITO", resultado: {numero_serie, id_tienda, fecha_venta}}
21
Anota la solicitud en la bitácora [BROKER]
22
Reenvía la respuesta (mismo idSolicitud)
23
alt
[Respuesta exitosa]
Muestra la confirmación, limpia el formulario
y recarga las listas
24
[Respuesta de error]
Traduce el código de error y ofrece reintentar
25
Cliente (Sistema 2)
Broker
Proveedor (Sistema 1)
MariaDB